SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I ***G. Puneeth*** **(192524221)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
G. Puneeth

QUESTION 1

An airport has five check-in counters serving passengers. Each passenger arrival is considered a process with varying arrival and service times. During peak hours, passengers with special assistance are given priority. Analyze the scheduling using FCFS and Priority Scheduling. Determine the waiting time and turnaround time for each passenger and identify which algorithm provides better service efficiency.

Airport Check-in Counters (FCFS vs Priority Scheduling)
Example

Passenger Arrival Time Service Time Priority

P1	0	5	3
P2	1	3	1
P3	2	4	2
P4	3	2	1
P5	4	6	4

FCFS
Order:
P1 → P2 → P3 → P4 → P5

Passenger	WT	TAT
P1	0	5
P2	4	7
P3	6	10
P4	9	11
P5	10	16

Average Waiting Time = 5.8
Average Turnaround Time = 9.8

Priority Scheduling
Order:
P1 → P2 → P4 → P3 → P5

Passenger	WT	TAT
P1	0	5
P2	4	7
P4	5	7
P3	8	12
P5	10	16

Conclusion
Priority Scheduling provides quicker service to passengers requiring special assistance. FCFS is fair but does not prioritize urgent passengers.

QUESTION 2

A hospital emergency room receives patients continuously. Critical patients must be treated immediately, while non-critical patients wait in a queue. Model this as a process scheduling problem using Priority Scheduling and Round Robin (time quantum = 5 minutes). Compare the response time for critical and non-critical patients.

Hospital Emergency Room Example

Patient	Arrival	Treatment Time	Priority
Critical1	0	8	1
Normal1	1	10	3
Critical2	2	6	1
Normal2	3	5	4

Priority Scheduling
Execution Order
Critical1 → Critical2 → Normal1 → Normal2
Response Time
Critical patients = Immediate
Normal patients = Higher waiting time

Round Robin (Quantum = 5)
Execution
Critical1 → Normal1 → Critical2 → Normal2 → Remaining bursts

Comparison

Priority Scheduling	Round Robin
Immediate treatment for critical patients	Equal CPU sharing
Best for emergencies	Better fairness
Possible starvation	No starvation

Conclusion
Priority Scheduling is the best choice for hospitals because emergency patients receive immediate treatment.

QUESTION 3

A cloud service provider receives ten computational tasks from clients. Each task requires different CPU burst times and arrives at different intervals. FCFS, SJF, and Round Robin scheduling to determine the average waiting time and CPU utilization. Recommend the best scheduling algorithm for the cloud environment.

Cloud Service Provider

Task	AT	BT
T1	0	8
T2	1	4
T3	2	9
T4	3	5

FCFS
Average WT = **8.75**

SJF
Order
T2 → T4 → T1 → T3
Average WT = **5.25**

Round Robin (q=4)
Average WT ≈ **7.5**

CPU Utilization
Since CPU never becomes idle
CPU Utilization = **100%**

Recommendation
SJF gives the minimum average waiting time and is recommended for cloud task scheduling.

QUESTION 4

A smart city uses sensors to detect vehicle density at an intersection. Each traffic lane is represented as a process competing for signal access. Design a scheduling strategy that minimizes vehicle waiting time. Compare the effectiveness of Round Robin and Priority Scheduling for emergency vehicles.

Smart City Traffic Signal

Processes

- **Lane A**
- **Lane B**
- **Lane C**
- **Ambulance Lane**
-

Priority
Emergency Vehicle = Highest

Round Robin
Each lane receives equal green time.

Advantages

- **Fair**
- **No starvation**

Disadvantages

- **Ambulance waits.**

Priority Scheduling
Emergency vehicles always receive green first.

Advantages

- **Minimum emergency delay**

Disadvantages

- **Busy lanes may starve.**

Best Strategy
Priority Scheduling with Aging.

QUESTION 5

During an online examination, thousands of students submit answers simultaneously. The server processes requests based on arrival times and submission priorities. Construct a process scheduling table for eight requests and calculate completion time, turnaround time, and waiting time using FCFS and SJF scheduling.

Online Examination Server

Sample Requests

Request	AT	BT
R1	0	4
R2	1	3
R3	2	5
R4	3	2
R5	4	6
R6	5	4
R7	6	3
R8	7	2

FCFS

Calculate
Completion Time (CT)

Turnaround Time
TAT = CT – AT

Waiting Time
WT = TAT − BT
Average WT ≈ 8.5

SJF
Execution
R4 → R8 → R2 → R7 → R1 → R6 → R3 → R5

Average WT ≈ 5.2

Conclusion
SJF improves response time.

QUESTION 6

A factory assembly line consists of multiple robotic arms, each performing a task. If one robot fails, the operating system must reschedule pending tasks. Analyze how process states (Ready, Running, Waiting, Terminated) change during task execution and explain how scheduling ensures continuous production.

Factory Assembly Line

Process States

Robot starts
Ready
↓

CPU allocates
Running
↓

Robot needs spare part
Waiting
↓

Part arrives
Ready
↓

Execution
Running
↓

Task finished
Terminated

Scheduling Benefit

- **Failed robot enters Waiting.**
- **Remaining robots continue execution.**
- **Prevents production stoppage.**
- **Maximizes CPU utilization.**

QUESTION 7

A video streaming service processes user requests for video buffering. Premium users are assigned higher priority than standard users. Given a set of user requests with arrival and burst times, implement Priority Scheduling and evaluate fairness, starvation, and throughput.

Video Streaming Service

Example

User	Arrival	Burst	Priority
Premium1	0	5	1
Premium2	1	4	1
Standard1	2	6	3
Standard2	3	5	3

Priority Scheduling
Execution
Premium1 → Premium2 → Standard1 → Standard2

Evaluation
Fairness

Medium
Starvation
Possible for standard users

Throughput
High because premium users finish quickly

Improvement
Use Aging so waiting standard users gradually gain higher priority.

QUESTION 8

A bank processes ATM withdrawals, deposits, and fund transfers concurrently. Each transaction is treated as a process with varying execution times. Using Round Robin Scheduling (time quantum = 4 ms), determine the execution order and calculate average turnaround and waiting times for ten transactions.

Bank ATM Transactions Example

Transaction	Burst Time
T1	5
T2	4
T3	6
T4	3
T5	5
T6	4
T7	7
T8	2
T9	3
T10	5

Time Quantum = 4 ms

Execution Order
Round 1
T1 → T2 → T3 → T4 → T5 → T6 → T7 → T8 → T9 → T10
Remaining bursts continue in the next rounds until completion.

Results
Average Waiting Time ≈ 12 ms
Average Turnaround Time ≈ 18 ms

Conclusion
Round Robin ensures fairness and good response time for concurrent banking transactions.

QUESTION 9

An autonomous vehicle executes multiple processes simultaneously: obstacle detection, GPS navigation, speed control, and lane detection. Identify the process states of each task at different time intervals and justify the use of preemptive scheduling in ensuring passenger safety.

Autonomous Vehicle

Processes

Process	State
Obstacle Detection	Running
GPS Navigation	Ready
Speed Control	Ready
Lane Detection	Waiting (Sensor)

When obstacle detected
Obstacle Detection preempts all others.

GPS → Ready
Speed Control → Ready
Obstacle Detection → Running

Why Preemptive Scheduling?

- **Fast emergency response.**
- **Prevents accidents.**
- **Ensures passenger safety.**
- **Real-time execution.**

Justification

Preemptive Scheduling immediately interrupts lower-priority tasks when obstacle detection becomes active, ensuring passenger safety.

QUESTION 10

A satellite control center monitors communication, navigation, power management, and telemetry systems. Communication tasks have the highest priority during emergencies. Simulate process execution using Priority Scheduling and analyze the impact of process starvation. Propose a solution using aging techniques to improve fairness.

Satellite Control Center
Example

Process	Priority
Communication	1
Navigation	2
Power Management	3
Telemetry	4

Priority Scheduling

Execution
Communication
↓
Navigation
↓
Power Management
↓
Telemetry

Problem
If communication requests continue arriving, telemetry may never execute.
This is called **starvation**.

Aging Solution
Increase waiting process priority over time.
Example
Telemetry
Priority 4
↓
After waiting
Priority 3
↓
Priority 2
↓
Priority 1
Eventually, telemetry receives CPU time.

Conclusion
Aging prevents starvation while maintaining high-priority emergency communication.

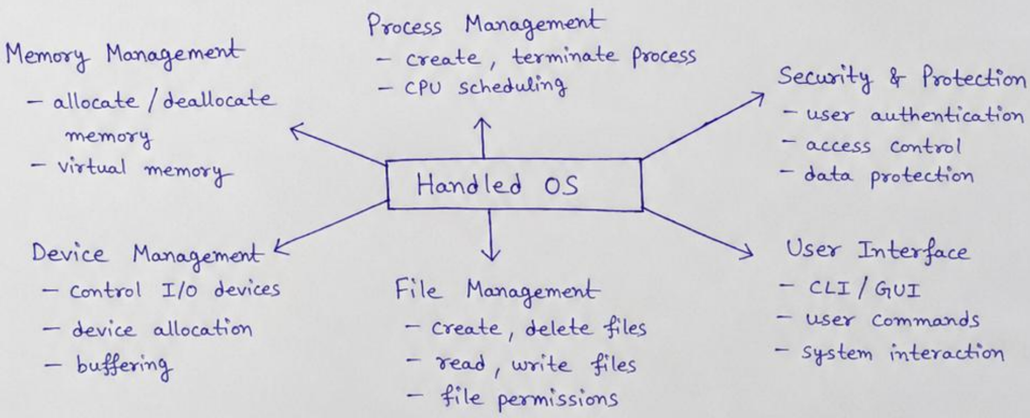
11.) Create a concept map highlighting the features and impact of handled OS.

Process State

As a Process executes, it changes state

- new : The Process is being created
- running
- waiting
- ready
- terminated

Features of OS



Impact of Handled OS

- Better Resource Utilization → Efficient use of CPU, memory, I/O
- Improved Performance → Reduced idle time, faster execution
- Security & Reliability → Protects data and system from unauthorized access
- Convenience → Provides easy interface and services to users
- Scalability → Supports multiple users and processes
- Overall System Efficiency → Smooth operation and better user experience

12) Consider the following processes :

Process	P_1	P_2	P_3	P_4
Burst Time (ms)	20	5	13	8

Time Quantum = 4 ms

a. Construct the Gantt chart for Round Robin scheduling.

b. Calculate the Waiting Time and Turnaround Time of each process.

c. Analyze how changing the time quantum to 10 ms would affect CPU utilization and context switching.

Solution :

a) Gantt chart for Round Robin Scheduling (Time Quantum = 4 ms)

P_1	P_2	P_3	P_4	P_1	P_2	P_3	P_4	P_1	P_3	P_1	
0	4	8	12	16	20	24	28	32	36	40	49

Step by Step Explanation :

1. 0-4 : P_1 executes for 4 ms (rem = 16)
2. 4-8 : P_2 executes for 4 ms (rem = 1)
3. 8-12 : P_3 executes for 4 ms (rem = 9)
4. 12-16 : P_4 executes for 4 ms (rem = 4)
5. 16-20 : P_1 executes for 4 ms (rem = 12)
6. 20-24 : P_2 executes for 1 ms (finishes)
7. 24-28 : P_3 executes for 4 ms (rem = 5)
8. 28-32 : P_4 executes for 4 ms (finishes)
9. 32-36 : P_1 executes for 4 ms (rem = 8)
10. 36-40 : P_3 executes for 4 ms (rem = 1)
11. 40-44 : P_1 executes for 4 ms (rem = 4)
12. 44-45 : P_3 executes for 1 ms (finishes)
13. 45-49 : P_1 executes for 4 ms (finishes)

b) Waiting Time and Turnaround Time

- Completion Time (CT) from Gantt Chart :

$P_1 \rightarrow CT = 49$

$P_2 \rightarrow CT = 20$

$P_3 \rightarrow CT = 45$

$P_4 \rightarrow CT = 32$

- Calculation Table :

Process	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT = CT - BT)	Waiting Time (WT = TAT - BT)
P_1	20	49	49 - 20 = 29	29 - 20 = 9
P_2	5	20	20 - 5 = 15	15 - 5 = 10
P_3	13	45	45 - 13 = 32	32 - 13 = 19
P_4	8	32	32 - 8 = 24	24 - 8 = 16

$$\text{Average Waiting Time (AWT)} = \frac{(9 + 10 + 19 + 16)}{4} = \frac{54}{4} = \boxed{13.5 \text{ ms}}$$

$$\text{Average Turnaround Time (ATT)} = \frac{(29 + 15 + 32 + 24)}{4} = \frac{100}{4} = \boxed{25 \text{ ms}}$$

c) Effect of changing Time Quantum to 10 ms

- With Time Quantum = 10 ms, each process will get more CPU time in one go.
- Number of context switches will decrease because preemptions are reduced.
- This reduces context switching overhead and improves CPU utilization.
- Waiting time may increase for short processes that arrive after long processes.
- Throughput may improve slightly due to fewer context switches.
- Overall, system becomes more efficient, but response time for some processes may increase.

13) A system has the following process behavior :

- Each process spends 20% of its time using the CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

The CPU utilization is given by :

$$\boxed{U = 1 - p^n}$$

where,

p = Probability that a process is waiting for I/O

n = Number of processes in memory

a) Calculate the CPU Utilization.

b) Compute the CPU Utilization if the degree of multiprogramming increases to 6.

c) Interpret which configuration provides better system performance and explain why.

Solution :

Given,

CPU time = 20%.

I/O waiting time = 80%.

So, p = Probability of waiting for I/O = 80% = 0.8

a) For degree of multiprogramming (n) = 4

Using
$$U = 1 - p^n$$
$$U = 1 - (0.8)^4$$
$$U = 1 - 0.4096$$
$$U = 0.5904$$
$$\boxed{U = 59.04\%}$$ (CPU Utilization)

b) For degree of multiprogramming (n) = 6
$$U = 1 - p^n$$
$$U = 1 - (0.8)^6$$
$$U = 1 - 0.262144$$
$$U = 0.737856$$
$$\boxed{U = 73.79\%}$$ (CPU Utilization)

c) Interpretation :

When the degree of multiprogramming increases from 4 to 6, CPU utilization increases from 59.04% to 73.79%.

This configuration with 6 processes provides better system performance because more processes keep the CPU busy most of the time while others wait for I/O, reducing CPU idle time and increasing overall utilization.